

Functional Programming Exam Part 2 of 2

Course responsible

David Sands (0737207663) will visit the exam rooms once at around 15:20–15:40, and is available by phone 16–17.30, and for students with extra time between 12:30 and 14:00.

Grading

There are two parts to this exam. This is Part 2. Part 2 will be graded only if you pass part 1, so we recommend that you do not submit Part 2 if you did not pass Part 1.

You should have completed Part 1 before starting Part 2. Because of limitations of the SafeBrowser system, you cannot have two exams open at the same time.

Grading scheme for Part 2:

Part 2 consists of small programming assignments with parts requiring a short written response. There are four questions of $8 + 8 + 6 + 8 = 30$ points.

- Chalmers: 22 points or more are required for a 5, and 14 points for a 4.
- GU: 22 points are required for a VG

Any bonus points (up to 4 points = 13%) will only be added if they make a difference your overall grade.

Your score in Part 1 does not influence your score on Part 2.

Notes

- If you give multiple answers to the same question only the first answer will be graded.
- For each part Question, if your solution consists of more than a few lines of Haskell code, use your common sense to decide whether to include a short comment to help your grader understand the solution.
- You can use any of the standard Haskell functions listed in the PDF accessible at the bottom of the SafeBrowser window.
- Full points are given to solutions which are short, elegant, and correct. Fewer points may be given to solutions which are unnecessarily complicated, unstructured, or repetitive.
- You are encouraged to use the solution to an earlier part of a Question to help solve a later part — even if you failed in solving the earlier part.

- 1 This question deals with the following data types for representing arithmetic expressions containing numbers, variables, addition and negation.

```
type VarName = String

data Exp = Add Exp Exp | Negate Exp | Var VarName | Num Int
  deriving (Eq, Show)

data EvalData = EvalData Exp [(VarName, Int)]
  deriving (Eq, Show)

data Simplified = Partial Exp | Full Int
  deriving (Eq, Show)
```

In this question you should define an evaluation function

```
eval :: EvalData -> Simplified
```

When given an input **EvalData exp table**, which is an expression and a lookup table for the values of variable names, **eval** computes the value of an expression if possible, returning a value of the form **Full n**.

If it is not possible (because there is a variable which is not in the table) it returns **Partial exp'** where **exp'** is a simplified version of **exp**

The simplifications you are required to perform are to make sure that

1. The only sub-expressions not containing variables are of the form **Num n**
2. There are no expressions of the form **Add e1 e2** where **e1 == Negate e2** or **e2 == Negate e1** (these should be replaced by **Num 0**)
3. Variables should be replaced with their values from the table whenever possible.

The data type invariant for **Simplified** is

```
prop_simplified :: Simplified -> Bool
prop_simplified (Partial (Num n)) = False
| | | -- should be represented as Full n
prop_simplified _ = True
```

Fill in your answer here. Do not type any of the above code.

- 2 This question is about an expected property of **eval**, and how it could be tested. Here are the definitions again:

```
type VarName = String

data Exp = Add Exp Exp | Negate Exp | Var VarName | Num Int
| deriving (Eq, Show)

data EvalData = EvalData Exp [(VarName,Int)]
| deriving (Eq, Show)

data Simplified = Partial Exp | Full Int
| | deriving (Eq,Show)
```

For any expression **exp**, if the table **t** contains enough variables, then the following property should hold:

```
prop_eval :: EvalData -> Bool
prop_eval evdata = case eval evdata of
| | | | | | | | | | Partial _ -> False
| | | | | | | | | | _ -> True
```

Make an Arbitrary instance definition for **EvalData** such that it can generate values **EvalData exp table** where **exp** can be of any size, and have any number of variables, but where **table** should contain values for all of the variables in **exp**.

Fill in your answer here. Do not type any of the above code.

1

3 Consider a function

```
trim :: [[a]] -> [[a]]
```

which trims items from the input lists so that they all have the same length (i.e. the length of the shortest one). For example:

```
prop_trim = trim [[1,2,3],[10,20,30],[100,200]] == [[1,2],[10,20],[100,200]]
```

This question is about lazy evaluation, and how different ways of defining trim affect what happens when one or more of the argument lists are infinite. We will consider the following two versions of trim:

```
trim1 xss = map (take (minLength1 xss)) xss
| | | where minLength1 = minimum . map length

trim2 xss = map (take (minLength2 xss)) xss
| | | where minLength2 = length . minimum . map (map $ const ())
```

Now consider the following three examples

```
ex1 = [[1,2,3],[1..]]
ex2 = [[1..]]
ex3 = [[1..n] | n <- [1..]]
```

- (A) What output we get when we run **trim1 ex1**? Explain why in one or two sentences.
 - (B) What output do we get when we run **trim2 ex1**. Explain why in one or two sentences.
 - (C) What output do we get when we run **trim2 ex2**. Explain why in one or two sentences.
 - (D) Explain why **trim2 ex3** does not produce any output*
(* actually ghci prints a single left square bracket)
 - (E) Explain why no definition of trim could do better on this input.
- Fill in your answer here**

- 4 One efficiency problem with lists is that the number of steps to append lists together depends on how we group the append operation. For example, $a \mathrel{++} (b \mathrel{++} c)$ takes $\text{length } a + \text{length } b$ append recursion steps, but $(a \mathrel{++} b) \mathrel{++} c$ takes $2 * \text{length } a + \text{length } b$ steps to produce the same result. In some recursive functions this property of append causes the complexity to go from linear to quadratic.

For situations where this is problematic then a different representation for lists can solve this problem. The basic idea is to represent a list of type $[a]$ as a function of type $[a] \rightarrow [a]$.

As an example, $[1,2,3]$ will be represented by a function equivalent to $\text{ys} \rightarrow 1:2:3:\text{ys}$. Note that to convert this to a regular list we just need to apply it to the empty list. But the benefit with this representation is that we can define append directly on this representation in such a way that the order in which we append several lists does not change the efficiency.

This idea is used inside the Show type class to create efficient show functions for recursive data types.

Here is a partial implementation of an abstract data type for lists using this idea. Your task is to complete the definition as specified below.

```
newtype DiffList a = DL ([a] -> [a])

nil :: DiffList a
nil = DL id

toList :: DiffList a -> [a]
toList (DL d) = d []

append :: DiffList a -> DiffList a -> DiffList a

cons :: a -> DiffList a -> DiffList a
```

In this question you should

- (A) [3 points] provide the missing definition of **append**,
- (B) [3 points] provide the missing definition of **cons**, and
- (C) [2 points] provide a suitable module definition (just the lines that should go at the top of the file) that make this into an abstract data type.

Hint: just focus on providing a simple and correct definition without trying to convert to regular lists and back again, and the desired efficiency aspects should take care of themselves. You should **NOT** define a **fromList :: [a] -> DiffList a** function to do this.

Fill in your answers to parts (A), (B), and (C) here